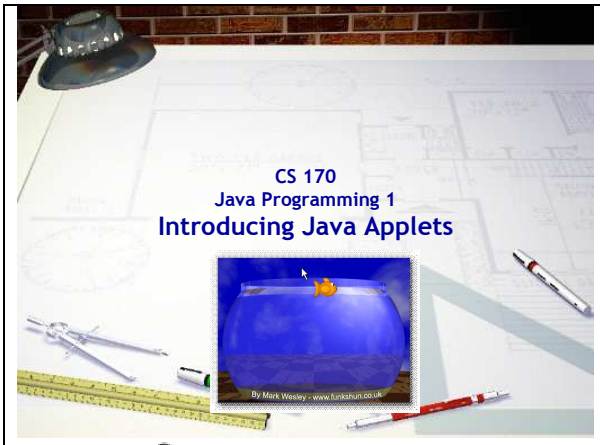


Slide 1

CS 170
Java Programming 1
Introducing Java Applets
Duration: 00:01:13
Advance mode: Auto

Notes:

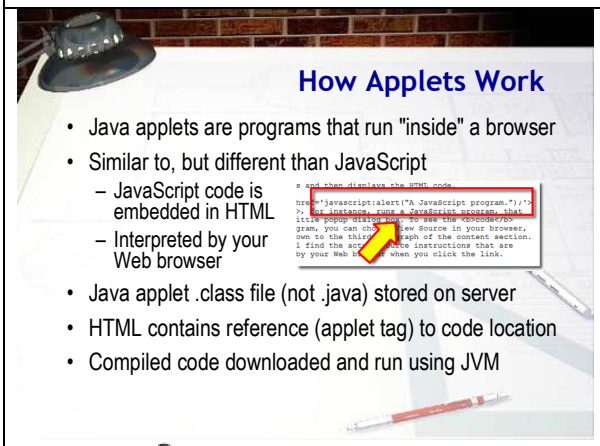
Hi there. This is the CS 170, Java Programming 1 lecture, introducing Java Applets

What's the purpose of a computer? That's pretty simple: computers are designed to run programs, just like your iPod is designed to play music, and your DVD player is designed to show movies.

To run a particular program on your computer, a spreadsheet, say, or a computer game, you can install the program on your local hard disk and start it running from your computer's "desktop", by clicking on its icon. Java, though, gives you another option.

Using Java you can write computer programs that run inside your Web browser, like Mark Wesley's "Fish Bowl" program shown here; with these kinds of programs, called Java applets, there's no separate download step and no explicit run step: you just visit a Web site containing the Java program, and it automatically starts working inside your browser.

So far, all of your programs have been text-based (console mode) Java applications. Let's look at writing your first graphical Java applet.



Slide 2

How Applets Work
Duration: 00:01:39
Advance mode: Auto

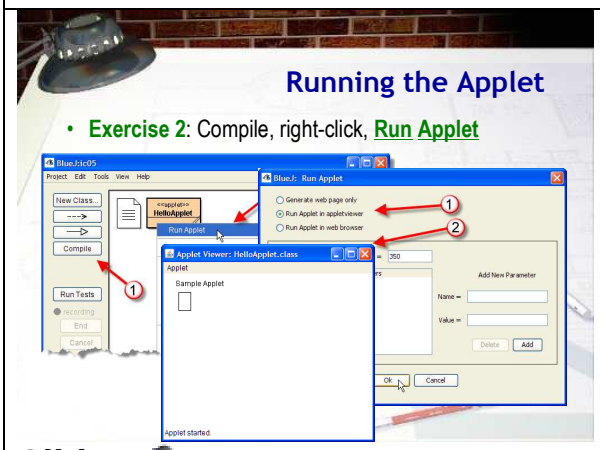
Notes:

Java applets are programs that run inside of your Web browser, kind of like JavaScript programs, but different. When you use a scripting language like JavaScript, the actual human-readable source code for your program is embedded along with the HTML in your Web page like this. When your browser downloads a page containing JavaScript, your browser itself interprets the JavaScript code and then executes it, much the same way it interprets and then displays the HTML code.

Java applets, on the other hand, work differently. An applet is a compiled Java program (that is, the .class file, not the .java file), that is stored on a Web server. The code for the applet itself isn't contained inside your Web page. Instead, you add a special tag to your HTML, (called the applet tag), that tells the Web browser where to find the compiled code, and how to display it.

When your Web browser encounters a reference to a Java applet in a Web page, it sends another, separate request to the server, asking for the compiled applet code. This is the same way that images or other types of "embedded" content, such as Flash movies, work.

A Java applet is a special kind of Java program, because it can't run on its own: it needs a Web browser to give it a "home". When your Web browser receives the compiled Java program from the Web server, it launches a Java Virtual Machine (this is the interpreter that will run the Java program),

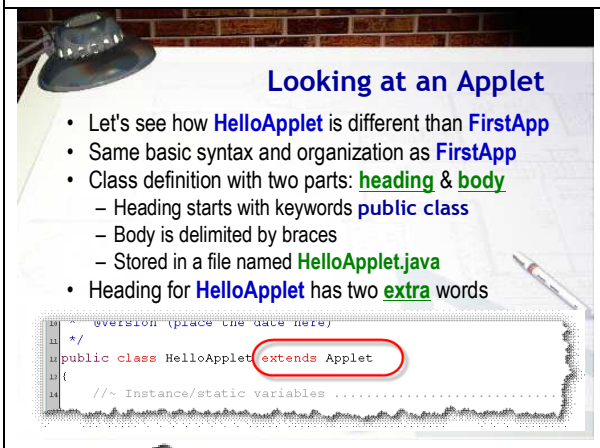
	provides a bit of screen real estate for the applet to use, and then lets the applet start running.
 **Slide 3** **Applets in BlueJ** Duration: 00:01:08 Advance mode: Auto	**Notes:** Creating applets in BlueJ is quite a bit easier than doing all of the steps manually. BlueJ will automatically: • create the skeleton for a basic Java applet • create a basic HTML page needed to display the applet • write the applet tag that embeds the applet in the page • and then launch either your Web browser or the appletviewer to run your program In this lecture, I'll walk you through the mechanics of creating a Java applet using BlueJ and then we'll compare the applet code to the first console-mode program that we wrote: FirstApp.java Start a new section in your in-class document, make sure BlueJ is open, and let's get started. For Exercise 1 complete these steps. • Click the New Class button to create a new class • Name the class HelloApplet and choose Java Applet from the Create New Class dialog • Open the new class in your editor and shoot me a picture of your applet opened in the editor.
 **Slide 4** **Running the Applet** Duration: 00:01:45 Advance mode: Auto	**Notes:** Before we take a look at the code, let's run the applet. Here are the steps to follow. • First, make sure your program compiles. Just click the Compile button on the project window, right-click the class and choose Compile from the context menu, or click the Compile button in the editor. • Next, right-click the class in the project window, and choose Run Applet from the context menu. • When the Run Applet dialog window appears, you'll be given a choice between generating a Web page (and not running the applet), generating the Web page and running the applet in the appletviewer program or generating the Web page and running the applet in your browser. Normally you'll choose to run the program in appletviewer. (Your Web browser's cache treats applets differently than

Web pages, so using your Web browser while developing applets can be really frustrating, because it's very easy to be editing and running different versions of your applet.)

- The Run Applet dialog also allows you to specify the size of the applet. Because applets borrow their screen real estate from the browser, the browser needs to know how much space it needs before you launch the applet; it can't resize itself afterwards. BlueJ defaults to 500 by 500 which I find kind of large. I usually reduce it somewhat.

Click the OK button, and the applet will run. For this exercise (Exercise 2), shoot me a screen-shot of the stock applet running in the applet viewer.

Now let's take a look inside our applet and see how its structure differs from a regular console program.



Slide 5

Looking at an Applet

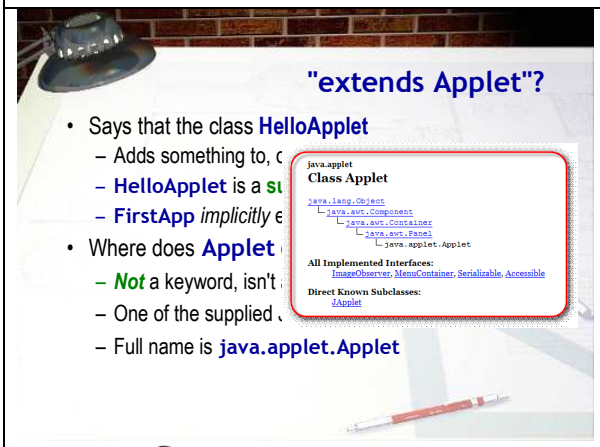
Duration: 00:01:01
Advance mode: Auto

Notes:

The HelloApplet probably looks familiar, but maybe a little strange as well. That's because while it's similar to the programs you've written so far, such as FirstApp it's not exactly the same.

Let's take a look at a few of the syntax issues that arise because of the differences between applets and applications. First, the program has the same basic syntax and organization as the application FirstApp. Like FirstApp it consists of a class definition with both a heading and body, the body is surrounded by braces and the source code is stored in a file named HelloApplet.java. All of that is the same.

What's different is that the class header for the applet HelloApplet, on line 12, is slightly more complex than that for the application programs you've written previously. It has two extra words at the end, which you haven't yet encountered. Let's look at those.



Slide 6

"extends Applet"?

Duration: 00:02:27
Advance mode: Auto

Notes:

So, what does extends Applet mean. The first word, extends, is a keyword like public and class. The extends keyword means that what precedes it (in this case the public class HelloApplet), adds something to, or builds upon, what follows it (in this case, something called Applet). This is the object-oriented feature known as inheritance, which we'll cover in greater depth later in the course. In technical object-oriented terms we way that HelloApp is a subclass of the superclass applet.

You might wonder why FirstApp.java doesn't have a similar extends clause tacked onto the end of its header line. Surprisingly, it does, in a way. If you don't explicitly specify a superclass when you write a class definition, Java implicitly tacks on the phrase extends Object to the end. You can, if you like, explicitly add extends Object, but you don't have to.

This sort of makes sense. If a header is supposed to introduce a class, this part of the header is describing its lineage. Sort of like a medieval herald at a ball:

Announcing Fred, Duke of URL, heir of Oingarten

The real question here is:

- "What's an Applet,
- and where does Applet come from?"

We can tell right away that Applet isn't a keyword like public or class or extends, because we know that all of the keywords are lowercase, and Applet is capitalized.

Perhaps, our knowledge of convention might help; and, indeed it does. If you recall the convention for identifiers, you'll see that only classes begin with a capital (like Applet) followed by lowercase characters. In fact, Applet is a class, one of the classes supplied as part of the Java Platform.

The Applet class actually has quite a pedigree, which you can discover by looking at the Java documentation (JavaDocs) for the Applet class. As you can see, the Applet class is descended from the Panel class, which is a child of the Container class, which is a subclass of Component, which comes from Object, the root or base class for all Java classes, (both the "built-in" classes, and the user-defined classes that you'll create on your own.)

Slide 7

Inside HelloApplet

Duration: 00:00:43

Advance mode: Auto

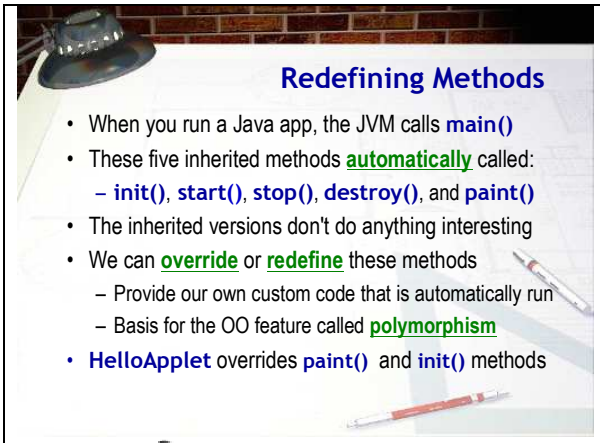
Notes:

The class body of an applet contains fields and methods, just like any other Java class. The template Applet class written by BlueJ contains a sample field (in case you forget how to define an instance variable.) We won't need that right now, so just delete it.

Now, let's look at the methods.

The first thing you'll probably notice, is that there is no main() method, like FirstApp had. Applets never have main() methods like applications. Instead, since HelloApplet is a "child" (or subclass) of Applet, it inherits certain methods from its "parent", grandparent and great-grandparent.

Let's see how that works.



Slide 8

Redefining Methods

Duration: 00:01:46
Advance mode: Auto

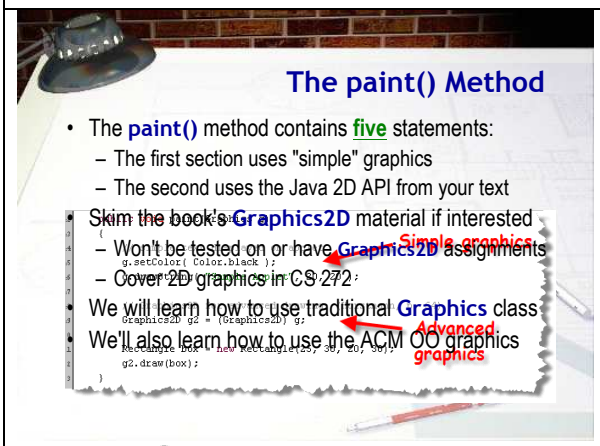
Notes:

When you run a Java application, the Java Virtual machine starts your program running by calling the main() method, automatically. Applets work similarly, except, instead of one method, there are actually five different inherited methods (that is, methods defined in the Applet or other superclasses) that are automatically called whenever an applet runs, instead of just one.

The five methods are:

- `init()`, which is called whenever your applet is first loaded into memory. This gives you a chance to initialize your instance variables, just like you would with a constructor in a regular class.
- `start()` which is called once your program is loaded into the browser and ready to go.
- `stop()`, called when the user navigates away from the page containing your browser
- `destroy()`, called whenever the browser unloads the code that contains your applet
- `paint()`, which is called any time the operating system determines that your applet needs to refresh the contents that appear on your applet's window.

Now, even though these methods are already defined in the Applet and Component classes, the versions defined there don't actually do anything interesting. In fact, they are completely empty. That seems kind of stupid, but it turns out to be really clever. What you can do is redefine any or all of these inherited methods, providing your custom code for your specific applet. Then, when the original method is automatically called, your replacement code will run instead. These redefined methods are said to be overridden, and this is the basis for the object-oriented feature known as polymorphism.



Slide 9

Notes:

In your editor, scroll on down and locate the paint() method. Notice that it contains five statements.

The first section, which sets the text color to black and prints the message "A Sample Applet" on the screen, uses the traditional "simple" or "original" graphics API. The second section, which draws a rectangle below the message, uses the more advanced Java 2D API from your text.

If you're interested in the Java Graphics2D material, you can skim your textbook's coverage. I think that it's a little too advanced and confusing for this point in the course, though, so I won't give you any Graphics2D assignments or ask any questions on the test. We will actually cover that material in

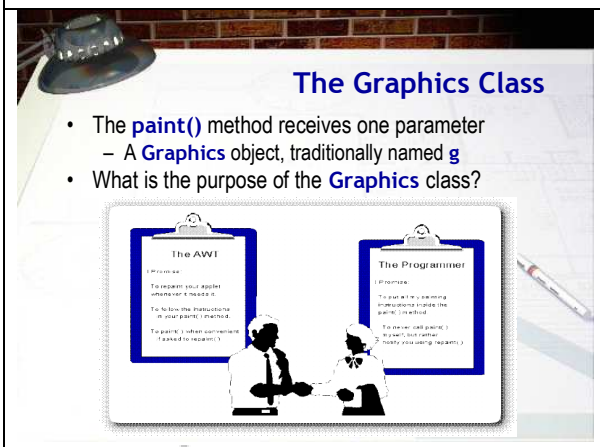
The paint() Method

Duration: 00:01:17
Advance mode: Auto

CS 272, the Java Programming II course.

In this class, we're going to create two kinds of graphics programs. First, we'll learn how to use the traditional Graphics class that works in all versions of Java. Second, we'll learn how to use some more advanced object-oriented Graphics classes that were designed especially for instruction by the Association of Computing Machinery.

Let's start by looking at the Graphics class. What is it, and what does it do?



Slide 10

The Graphics Class

Duration: 00:03:07
Advance mode: Auto

Notes:

The applet's paint() method receives one parameter, the Graphics object named "g". Who, or what, exactly, is "g"; what is the purpose of the Graphics class?

As you might have guessed by reading through the code, the drawString() method uses the Graphics object to draw on the display of an applet. You might wonder why this is. Why can't you just tell your computer to paint a red pixel at location 100, 100, without involving some Graphics object?

In the "old days", back when every computer ran a single program at a time, telling your computer to turn a particular pixel red was a reasonable request. Today, however, it doesn't make much sense at all. Here's why.

As I'm writing this, I have fourteen different applications running in the foreground. If all fourteen of my applications decided to paint at location 100, 100 at the same time, chaos would quickly follow. Obviously, with multiple applications running, each application cannot treat the display as its own private resource. Each one has to share with the others. They all have to agree to abide by the same "contract between the AWT (or Abstract Windowing Toolkit) and you, the programmer.

Let's take a look at how this programmer-AWT contract works.

The key figure in this picture is the AWT (the Abstract Window Toolkit) and its Graphics object. The Graphics object acts as a "traffic cop" to make sure your efforts to paint on the display don't interfere with other programs, or with different parts of your own program.

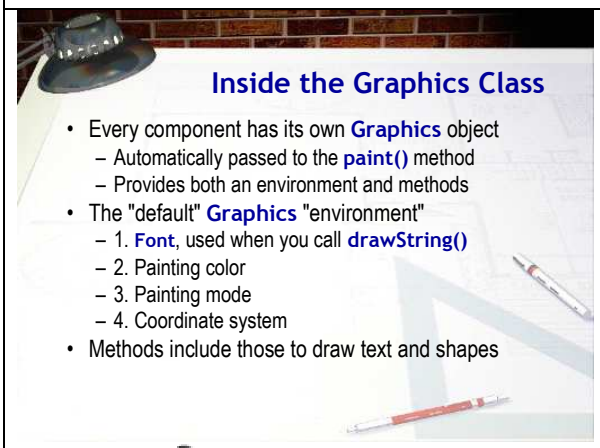
It does this in three ways:

1. The Graphics object "clips" your drawing region. If you create a drawing area that is 100 pixels square, and you try to paint a bright red circle that extends outside your drawing area, your Graphics object, in effect, holds a "stencil", (called the clipping region) over your drawing area so your efforts don't spill over on your neighbor's area.
2. The AWT "monitors" your drawing region. If another application should drag a dialog box over your drawing area, or if your application should scroll a portion of your image out of its window, the AWT will "notify" you when

your drawing region needs to be repainted. (It notifies you by calling your applet's paint() method.

3. The AWT "schedules" access to your drawing region, so that different methods don't end up fighting over the area. Although you have to write the instructions (inside your paint() method), that do the actual painting, you can ask the AWT to perform the painting, by calling your applet's repaint() method. This submits a "work order" with the AWT, letting it know that you'd like to have your applet repainted.

The AWT manages the overall painting process, but it doesn't write the instructions that do the actual painting. That responsibility is up to you. To do that, you'll use the Graphics class.



Slide 11

Inside the Graphics Class

Duration: 00:01:46
Advance mode: Auto

Notes:

Every Java Component has its own Graphics object. A copy of that Graphics object (not the original) is passed to the paint() method when it is called. When you receive your Graphics object, you're going to use it for two things:

- The Graphics object provides the environment in which you'll work: your own "painting studio" to speak. There are several methods that allow you to modify that environment.
- The Graphics object contains a set of methods that will help you in your actual painting. Think of these as your "painter's apprentice." You can use these methods to do your drawing.

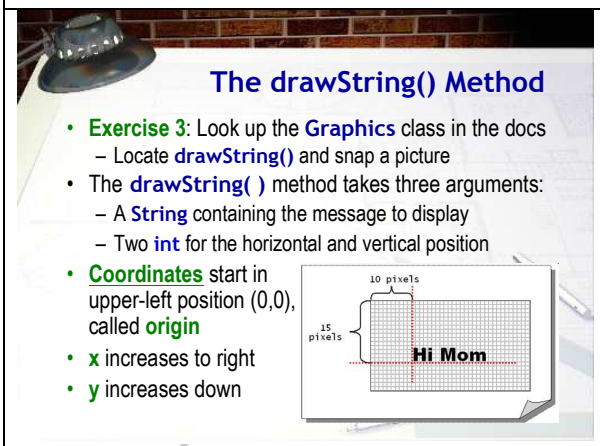
Let's look at those two parts:

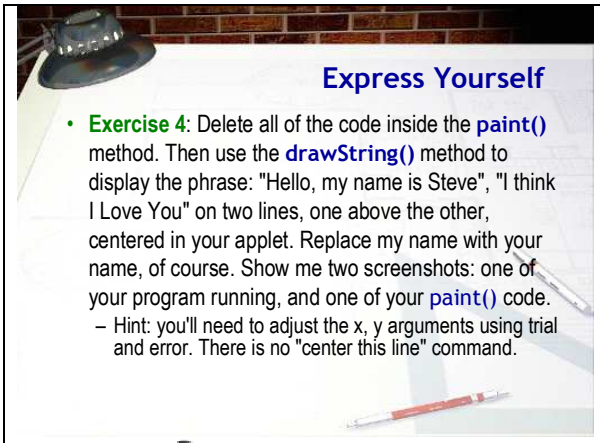
The Graphics object contains the "environment" used when any Java graphical component is drawn. This environment consists of:

- a default Font, used when you call methods like drawString() .
- a default "brush", used when painting lines or shapes.
- a default painting Color object.
- a default painting mode.
- a default coordinate system.

The Graphics object that you receive, itself "borrows" several of these items from the graphical component it represents. A Graphics object uses its component's foreground property as its default painting color, for instance, and the Graphics coordinate system is relative to the upper-left corner of its component.

Inside your paint() method you can make changes to the temporary Graphics copy you receive by calling the methods in the Graphics class, like our sample applet does when it changes the font color using the setColor() method.

	In addition, there are methods to display text and shapes. Let's look at the one shown here, drawString().
 **Slide 12** **The drawString() Method** Duration: 00:01:52 Advance mode: Auto	**Notes:** When you write a Java applet, instead of using System.out.println() to print a message on the console window, like you did with FirstApp, you'll draw your text on the surface of your applet by using the Graphics drawString() method. For Exercise 3, locate the Graphics class in the JavaDocs, then, locate the drawString() method documentation and snap a picture. As you can see, using drawString() is a little more complicated than using System.out.println(). With System.out.println(), you only needed to supply the text you wanted to display. With the drawString() method, however, you need to provide three arguments (or parameters): • the text you want to display • the horizontal position to start drawing • the veridical position to start drawing To display the text "Hi Mom" 15 pixels down from the top of your applet and 10 pixels in from the left you'd need to write g.drawString("Hi Mom", 10, 15); When drawing text, the horizontal (x), and vertical (y) positions where the text appears are measured from the upper-left corner of your applet, (not from the upper-left corner of your screen or of your browser window). • The unit used for measurement is the pixel • The x,y arguments represent the baseline of the text--an imaginary line where the text "sits". Note, since the coordinates used by drawString() represent the baseline of the text, if you use the coordinates 0,0, all of your text will be positioned outside of your applet, because it will be sitting "on top of" the top border. Java (wisely) prohibits you from painting all over screen real estate that you don't own, so you won't be able to see your text at all.



Slide 13

Express Yourself

Duration: 00:01:10
Advance mode: Auto

Notes:

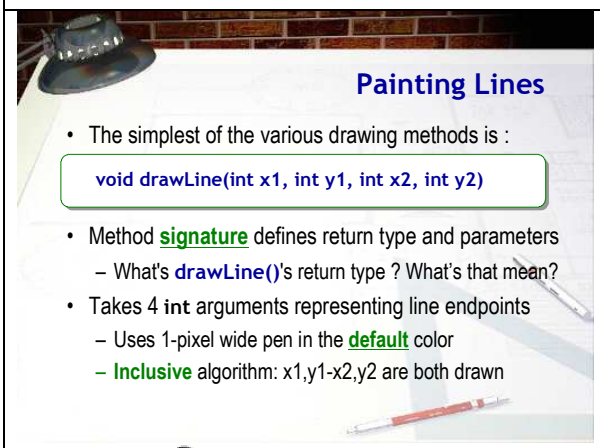
OK, enough talking from me. Now it's your turn. Let's change the HelloApplet so that the message is a little closer to what we used in FirstApp. Start by deleting all of the code inside the paint() method. (Make sure you leave the braces, though.)

Then, use the drawString() method to display the phrase "Hello, my name is Steve" on one line and "I think I love you on the second line". The lines should appear one above the other and each line should be centered horizontally in the applet window. The pair of lines should be centered vertically. (That is, you won't have extra space between the two lines, but you will have equal space above the first line and below the second.)

Oh, and make sure you replace my name with yours, of course.

Show me two pictures: one of your program running, and another of your code.

Here's a hint to get you started. There is no "Center this line" command with drawString(). Instead, you'll need to manually adjust the x and y parameters by trial and error.



Slide 14

Painting Lines

Duration: 00:02:33
Advance mode: Auto

Notes:

In addition to displaying text, one "role" of the Graphics object passed into the paint() method, is to provide methods that act as "artist's assistants." These methods draw lines, simple shapes, and complex geometric patterns. As you'll see later, these assistants know how to work with JPEGs and GIFs as well.

The simplest of these methods is the one that draws lines. Let's take it for a spin, right now.

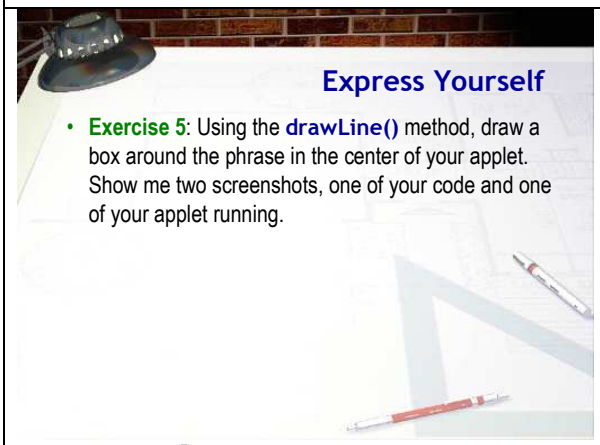
The drawLine() method takes four int arguments, representing the two end-points of a line. Here is the method signature:

void drawLine(int x1, int y1, int x2, int y2)

By the way, the word signature is how we refer to the documentation of the method parameters and return type like this. Once you know the signature, you'll know how to call the method. For instance, here, the return type of drawLine() is void. What does that mean? It means that the method doesn't return a value, but just carries out an action.

The parameters x1,y1 represent the horizontal (x), and vertical (y) point at one end of the line, while x2,y2 represent the point at the other end. Since all four arguments are int, you simply substitute a whole number for each argument when calling the method.

When you draw lines in Java, all lines are 1 pixel thick, and use the default Graphics brush color (retrieved from your component's foreground property). You can change the color of the brush, used to paint the lines, but you can't change the

	thickness of the line. Java actually can draw fancier lines with different thicknesses, styles and end-types. To do so, though, you have to use the Java 2D package and the Graphics2D class, which is slightly more complex than what I've shown you here. Once you've learned how to use the simpler Graphics class, you'll find the Graphics2D much easier to understand. The algorithm used to draw lines in Java is inclusive; both the point x1,y1 and the point x2,y2 are included in the line. Thus, if you want to draw a single pixel on the screen--at point 100,100, for instance, you do it like this: g.drawLine(100, 100, 100, 100);
 **Slide 15** **Express Yourself** Duration: 00:00:26 Advance mode: Auto	**Notes:** Let's finish up this week's graphics interlude by giving you a chance to take drawLine() out for a spin. Using only the drawLine() method, draw a box around the phrase you wrote in Exercise 4. Show me two screen-shots: one of your code, and one of your applet running.

